

ECOTRACK

Document de Cadrage Technique (DCT)

Plateforme intelligente de gestion des déchets urbains

Mastère EADL — Expert en Architecture et Développement Logiciel

Certification RNCP 38822 — Filière Développement

Projet fil rouge · École INGETIS · Année 2025-2026

Candidat	Remi Fauchon / Poilly Loris
Version du document	1.0
Périmètre	Application full-stack (web)
Dépôt de code	app/ (monorepo backend NestJS + frontend React)
Documentation API	OpenAPI/Swagger — /api/docs
Application en ligne	https://ecotrack.lorisdev.fr

Sommaire

0. Introduction

1. Contexte et objectifs

2. Périmètre fonctionnel

3. Architecture technique

4. Stack technologique

5. Modèle de données

6. Conception des API REST

7. Sécurité

8. Optimisation des tournées (TSP)

9. Temps réel et chaîne IoT

10. Gamification et engagement citoyen

11. Stratégie de tests et qualité

12. Infrastructure, conteneurisation et déploiement

13. Exigences non fonctionnelles (NFR)

14. Gestion de projet

15. Conclusion et perspectives

Annexes : A. Architecture — B. Modèle de données (ERD) — C. API (OpenAPI) — D. Captures — E. Glossaire.

0. Introduction

Ce Document de Cadrage Technique (DCT) formalise la conception et la réalisation d'ECOTRACK, une plateforme logicielle de gestion intelligente des déchets urbains, développée dans le cadre du projet fil rouge de Mastère 2 (filière Développement). À la différence d'un cahier des charges, qui exprime un besoin, ce document décrit une solution effectivement construite : il s'appuie sur le code réellement produit, sur la documentation d'API générée automatiquement et sur les résultats de tests mesurés.

Le lecteur visé est double : l'équipe pédagogique et le jury de certification, d'une part, qui y trouveront la justification des choix d'architecture et de leur mise en œuvre ; un développeur rejoignant le projet, d'autre part, qui pourra s'en servir comme document d'onboarding technique. Les extraits de code présentés sont issus du dépôt et illustrent les points de conception les plus structurants.

Le document suit une progression du général au particulier : contexte et périmètre, puis architecture et stack, puis chaque préoccupation transverse (données, API, sécurité, algorithmique, temps réel, qualité, déploiement), pour conclure sur les exigences non fonctionnelles et les perspectives d'évolution.

1. Contexte et objectifs

1.1 Contexte métier

Une métropole de 500 000 habitants gère un parc de 2 000 conteneurs à déchets répartis sur 12 secteurs géographiques. La collecte est aujourd'hui planifiée à fréquence fixe, indépendamment du remplissage réel des conteneurs. Cette approche calendaire engendre trois dysfonctionnements : des passages inutiles (30 à 40 % des arrêts portent sur des conteneurs quasi vides), des débordements sur les points les plus sollicités, et une absence de données exploitables pour piloter le service. Il en résulte des coûts opérationnels élevés (carburant, main-d'œuvre, maintenance) et un impact environnemental évitable.

1.2 Parties prenantes

Partie prenante	Attentes
Métropole (commanditaire)	Retour sur investissement, efficacité, conformité réglementaire
Services techniques (gestionnaires)	Outils de pilotage et d'optimisation
Agents de collecte	Application de terrain intuitive, itinéraires clairs
Citoyens	Service de qualité, moyen d'agir et d'être reconnus
DSI	Sécurité, disponibilité, maintenabilité

1.3 Solution proposée

ECOTRACK fait passer la collecte d'une logique calendaire à un pilotage par la donnée temps réel. Chaque conteneur est équipé d'un capteur ultrasonique transmettant une mesure toutes les 15 minutes via le protocole MQTT (de l'ordre de 500 000 messages par jour à l'échelle cible). Ces données alimentent une cartographie temps réel, un moteur d'optimisation des tournées (algorithme du voyageur de commerce raffiné par 2-opt) et un volet participatif citoyen reposant sur la gamification.

1.4 Objectifs et indicateurs cibles

Objectif	Indicateur	Cible
Optimiser les tournées	Réduction des distances	-20 %
Réduire les débordements	Taux de débordement	< 2 %
Économiser	Coûts opérationnels	-15 %
Décarboner	Émissions de CO ₂	-18 %
Fiabiliser	Disponibilité plateforme	> 99,5 %
Engager	Citoyens actifs	15 000

1.5 Périmètre et hors-périmètre

Le présent projet couvre l'intégralité de la couche applicative (filière Développement) : interfaces web, API, base de données, algorithmique, chaîne d'ingestion IoT simulée et sécurité applicative. Les couches réseau et infrastructure physique (segmentation, pare-feu, supervision système) relèvent de la filière Cyber-Réseaux ; l'analytique avancée et les modèles prédictifs relèvent de la filière Data. Ces interfaces sont mentionnées lorsqu'elles conditionnent la conception applicative.

2. Périmètre fonctionnel

2.1 Acteurs

Acteur	Volume	Rôle
Citoyen	15 000	Signale les conteneurs pleins, suit ses points/badges, participe aux défis
Agent de collecte	50	Reçoit sa tournée, valide les collectes (scan QR), signale les anomalies
Gestionnaire	10	Pilote en temps réel, optimise les tournées, génère les rapports
Administrateur	3	Gère les utilisateurs, configure les seuils et canaux

2.2 Cas d'utilisation couverts

Code	Cas d'utilisation	État
UC-C01	Signalement de conteneur plein (+ photo, +10 points)	Implémenté
UC-C02	Historique personnel (points, badges, CO ₂ évité)	Implémenté
UC-C03	Participation à un défi collectif	Implémenté
UC-A01	Réception de la tournée du jour	Implémenté
UC-A02	Validation d'une collecte (scan QR, volume, GPS)	Implémenté
UC-A03	Signalement d'anomalie terrain	Implémenté
UC-G01	Tournée optimisée (seuil paramétrable)	Implémenté
UC-G02	Monitoring temps réel	Implémenté
UC-G03	Rapport mensuel (PDF)	Implémenté
UC-AD01	Gestion des utilisateurs et rôles	Implémenté
UC-AD02	Configuration des alertes (seuils, canaux)	Implémenté
UC-T01	Authentification sécurisée (MFA, verrouillage)	Implémenté
UC-T02	Réception des données IoT (MQTT)	Implémenté

2.3 Catalogue détaillé des cas d'utilisation

Les quatorze cas d'utilisation sont décrits ci-dessous selon un format uniforme (acteur, précondition, scénario nominal, exceptions, post-condition). Tous sont implémentés et vérifiés dans la version livrée.

UC-C01 — Signalement d'un conteneur plein

Acteur : Citoyen authentifié

Précondition : Application ouverte, utilisateur connecté

Scénario nominal :

1. Le citoyen localise le point concerné sur la carte (clic / géolocalisation).

2. Il choisit le type de problème (conteneur plein, dépôt sauvage, conteneur endommagé).
3. Il joint éventuellement une photo et un commentaire.
4. Il valide ; le système enregistre le signalement avec horodatage et position.
5. Le système crédite 10 points, fait progresser les défis rejoints et notifie le gestionnaire de zone en temps réel.

Exceptions : Doubleton récent (< 1 h) sur le même point ; position hors zone couverte.

Post-condition : Signalement enregistré, points crédités, gestionnaire notifié.

UC-C02 — Consultation de l'historique personnel

Acteur : Citoyen authentifié

Précondition : Utilisateur connecté

Scénario nominal :

1. Le citoyen accède à son espace.
2. Il consulte ses points cumulés et ses badges (débloqués / à débloquent).
3. Il visualise l'estimation de CO₂ évité et la liste de ses signalements (avec statut).
4. Il consulte sa position au classement.

Exceptions : Aucun historique si le citoyen n'a jamais agi (affichage d'un état vide).

Post-condition : Le citoyen dispose d'une vue synthétique de sa contribution.

UC-C03 — Participation à un défi collectif

Acteur : Citoyen authentifié

Précondition : Au moins un défi actif

Scénario nominal :

1. Le citoyen consulte les défis collectifs actifs et leur progression.
2. Il rejoint un défi.
3. Chacun de ses signalements ultérieurs incrémente la progression collective et sa contribution personnelle.
4. À l'atteinte de l'objectif, chaque participant reçoit la récompense en points.

Exceptions : Défi déjà rejoint (action idempotente) ; défi expiré.

Post-condition : Participation enregistrée ; récompense distribuée à l'atteinte de l'objectif.

UC-A01 — Réception de la tournée du jour

Acteur : Agent de collecte

Précondition : Tournée planifiée et assignée

Scénario nominal :

1. L'agent s'authentifie.
2. Le système affiche ses tournées avec l'itinéraire tracé sur la carte.
3. Il consulte le détail : nombre d'arrêts, distance, durée estimée, statut.

Exceptions : Aucune tournée planifiée (liste vide).

Post-condition : L'agent dispose de sa feuille de route.

UC-A02 — Validation d'une collecte

Acteur : Agent en tournée

Précondition : Tournée en cours

Scénario nominal :

1. L'agent arrive au conteneur et scanne son QR code (ou saisit son code).
2. Le système identifie l'arrêt correspondant dans la tournée.
3. L'agent renseigne le volume collecté ; la position GPS est capturée.
4. Le système vide le conteneur (nouvelle mesure basse → état OK, alertes résolues, carte mise à jour).
5. La tournée est clôturée automatiquement lorsque tous les arrêts sont collectés.

Exceptions : Conteneur hors tournée ; arrêt déjà collecté ; géolocalisation refusée (repli).

Post-condition : Collecte enregistrée (volume, GPS), conteneur vidé, tournée mise à jour.

UC-A03 — Signalement d'une anomalie

Acteur : Agent de collecte

Précondition : Utilisateur connecté

Scénario nominal :

1. L'agent constate une anomalie (conteneur endommagé, accès bloqué).
2. Il saisit une description et envoie ; sa position est géolocalisée.
3. Le système enregistre l'anomalie et la transmet au gestionnaire (sans crédit de points).

Exceptions : Géolocalisation refusée (repli sur position par défaut).

Post-condition : Anomalie transmise au gestionnaire.

UC-G01 — Création d'une tournée optimisée

Acteur : Gestionnaire

Précondition : Données conteneurs à jour

Scénario nominal :

1. Le gestionnaire fixe un seuil de remplissage minimal à collecter.
2. Le système sélectionne les conteneurs concernés (requête PostGIS).
3. L'optimiseur calcule l'itinéraire le plus court (TSP + 2-opt) depuis le dépôt.
4. Le système restitue la tournée (ordre, distance, durée) et l'événement est diffusé en temps réel.
5. Le gestionnaire peut assigner la tournée à un agent.

Exceptions : Aucun conteneur au-dessus du seuil (message explicite, code 400).

Post-condition : Tournée optimisée créée et disponible.

UC-G02 — Monitoring temps réel

Acteur : Gestionnaire

Précondition : Utilisateur connecté

Scénario nominal :

1. Le gestionnaire consulte le tableau de bord et la carte.
2. Les conteneurs sont colorés par état ; les KPIs sont affichés.
3. À chaque mesure, la carte et les alertes se mettent à jour en temps réel (WebSocket).

Exceptions : Perte de connexion temps réel (rechargement périodique de secours).

Post-condition : Le gestionnaire dispose d'une vue temps réel de la métropole.

UC-G03 — Génération de rapport mensuel

Acteur : Gestionnaire

Précondition : Utilisateur connecté

Scénario nominal :

1. Le gestionnaire sélectionne un mois.
2. Le système agrège les indicateurs de la période (tournées, distances, alertes, signalements, mesures).
3. Le gestionnaire exporte le rapport en PDF.

Exceptions : Période sans activité (rapport à zéro).

Post-condition : Rapport mensuel généré et exportable.

UC-AD01 — Gestion des utilisateurs et des rôles

Acteur : Administrateur

Précondition : Compte administrateur

Scénario nominal :

1. L'administrateur accède à l'écran d'administration.
2. Il consulte la liste des utilisateurs.
3. Il modifie le rôle d'un utilisateur (Citoyen, Agent, Gestionnaire, Administrateur).

Exceptions : Tentative d'accès par un rôle non-administrateur (403 refusé).

Post-condition : Rôles mis à jour, appliqués aux prochaines connexions.

UC-AD02 — Configuration des alertes

Acteur : Administrateur

Précondition : Compte administrateur

Scénario nominal :

1. L'administrateur définit les seuils par défaut (warning, critique).
2. Il active/désactive les canaux de notification (email, push, SMS).
3. Il peut ajuster les seuils d'un conteneur particulier.

Exceptions : Valeurs hors bornes (rejet par validation).

Post-condition : Paramètres enregistrés et appliqués.

UC-T01 — Authentification sécurisée

Acteur : Tous les utilisateurs

Précondition : Compte existant

Scénario nominal :

1. L'utilisateur saisit email et mot de passe.
2. Le système vérifie les identifiants (bcrypt) et l'éventuel verrouillage.
3. Si la MFA est activée (gestionnaire/admin), un code TOTP est demandé en seconde étape.
4. Le système délivre une paire de jetons JWT (access + refresh).

Exceptions : Mot de passe erroné (incrément des échecs ; verrouillage 15 min au 5e) ; code MFA invalide ; compte verrouillé (403).

Post-condition : Utilisateur authentifié, session établie.

UC-T02 — Réception des données IoT

Acteur : Capteurs IoT (2 000 conteneurs)

Précondition : Broker MQTT disponible

Scénario nominal :

1. Le capteur mesure le niveau de remplissage.
2. Il publie la trame (containerId, fillLevel, température, batterie, horodatage) sur le topic MQTT.
3. Le service d'ingestion consomme, valide et enregistre la mesure.
4. L'état du conteneur est recalculé, les alertes sont déclenchées si nécessaire, et le dashboard est mis à jour.

Exceptions : Trame invalide (ignorée et journalisée) ; conteneur inconnu.

Post-condition : Mesure persistée, état et alertes à jour, diffusion temps réel.

3. Architecture technique

ECOTRACK adopte une architecture applicative en couches, organisée en modules métier au sein d'une API NestJS, exposée à un client web React. Le découpage modulaire isole les domaines (inspiration Domain-Driven Design) et prépare une extraction ultérieure en microservices physiques (perspective). Le schéma global figure en Annexe A.

3.1 Vue d'ensemble en couches

- Couche présentation : application web React (SPA) — tableau de bord gestionnaire, espace citoyen, vue agent, écran administrateur.
- Couche métier : API NestJS exposant une API REST documentée et un canal temps réel WebSocket (Socket.io), organisée en modules à responsabilité unique.
- Couche données : PostgreSQL 15 + PostGIS (données métier et géospatiales) ; Redis (cache, rate limiting).
- Couche ingestion : broker MQTT (Mosquitto) alimenté par un simulateur de capteurs et consommé par un service d'ingestion.
- Couche infrastructure : conteneurisation Docker, orchestration Docker Compose (5 services).

3.2 Découpage en modules NestJS

Module	Responsabilité
auth	Authentification JWT, refresh, MFA (TOTP), verrouillage
users	Comptes, rôles (RBAC), points
zones	Secteurs géographiques
containers	Conteneurs, requêtes géospatiales PostGIS, calcul d'état
measurements	Ingestion des mesures, déclenchement d'alertes
alerts	Cycle de vie des alertes (déduplication, résolution)
routes	Optimisation et suivi des tournées (TSP)
signalements	Signalements citoyens et anomalies agents
gamification	Points, badges, classement
challenges	Défis collectifs
reports	KPIs et rapport mensuel
settings	Paramètres globaux (seuils, canaux)
iot	Simulateur + ingestion MQTT
realtime	Passerelle WebSocket

3.3 Cycle de vie d'une requête

Chaque requête HTTP traverse une chaîne de traitement standardisée par NestJS : le guard de limitation de débit, le guard d'authentification (validation du JWT), le guard de rôles (RBAC), le pipe de validation des données (DTO), puis le contrôleur qui délègue au service, lequel manipule les dépôts TypeORM. Les préoccupations transverses sont ainsi centralisées en façade.

src/main.ts — configuration globale de l'API

```
// Démarrage de l'application : préfixe global, validation stricte, Swagger
app.setGlobalPrefix('api', { exclude: ['health'] });
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // rejette les champs non déclarés
  forbidNonWhitelisted: true,
  transform: true,          // typage automatique des DTO
}));
app.enableCors({ origin: true, credentials: true });
```

3.4 Patterns de conception

- Repository : l'accès aux données est abstrait par les dépôts TypeORM, ce qui isole la logique métier du SGBD et permet de tester les services avec des dépôts simulés.
- Strategy : le moteur d'optimisation expose une fonction unique, implémentée par l'heuristique gloutonne puis le raffinement 2-opt — algorithmes interchangeables sans impacter l'appelant.
- Observer / event-driven : les événements métier sont propagés en temps réel via WebSocket ; l'ingestion IoT est asynchrone via MQTT.
- Injection de dépendances : native à NestJS, elle découple les composants et facilite les tests.

3.5 Décisions d'architecture (ADR)

Les principales décisions de conception sont synthétisées ci-dessous sous forme d'Architecture Decision Records (décision, justification, alternatives écartées).

Décision	Justification	Alternatives écartées
Monolithe modulaire (NestJS) plutôt que microservices physiques	Complexité opérationnelle maîtrisée au niveau M1 ; découpage en modules qui prépare une extraction ultérieure	Microservices physiques (surcoût infra, observabilité)
NestJS plutôt qu'Express nu	Structure imposée (modules, providers, guards), injection de dépendances, support natif Swagger/validation	Express (trop permissif), Fastify seul
TypeORM plutôt que Prisma	Support mature des types géométriques PostGIS et des requêtes spatiales	Prisma (support PostGIS limité, requêtes brutes nécessaires)
MQTT (Mosquitto) plutôt que Kafka	Standard IoT léger, suffisant pour le volume M1 ; intégration simple	Apache Kafka (retenu comme évolution)
PostgreSQL + PostGIS plutôt que MongoDB	Besoin relationnel fort + requêtes géospatiales natives (index GiST)	MongoDB (géo plus limité, pas de transactions relationnelles)
JWT access + refresh avec rotation	Sans état, scalable ; révocation possible via hash du refresh stocké	Sessions serveur (état partagé, moins scalable)
Leaflet + OpenStreetMap plutôt que Mapbox	Open source, sans clé API ni quota, suffisant pour le besoin	Mapbox GL (clé API, facturation à l'usage)

4. Stack technologique

Les choix techniques, arrêtés à l'issue de la veille comparative (dossier Bloc 1), privilégient des briques open source matures, un langage unique (TypeScript) côté client et serveur, et l'adéquation au besoin géospatial et temps réel.

Couche	Technologie (version)	Justification
Frontend	React 18 + TypeScript + Vite + TailwindCSS	Écosystème, typage, productivité, rendu rapide
Cartographie	React-Leaflet + OpenStreetMap	Open source, sans clé API
Backend	NestJS 10 + TypeScript	Modularité, injection de dépendances, « microservices-ready »
ORM	TypeORM 0.3	Support natif des types géométriques PostGIS
Base de données	PostgreSQL 15 + PostGIS 3.4	Relationnel + géospatial (index GiST)
Cache	Redis 7	Cache, sessions, compteurs de rate limiting
IoT	MQTT (Mosquitto 2)	Protocole léger standard de l'IoT
Temps réel	Socket.io 4	Diffusion d'événements vers le dashboard
Sécurité	JWT, bcrypt, otplib (TOTP)	Standards d'authentification et de hachage
Qualité	Jest 29, ESLint, Prettier	Tests automatisés, qualité de code
Conteneurisation	Docker + Docker Compose	Reproductibilité, orchestration

Le choix d'un langage unique (TypeScript) sur l'ensemble de la chaîne réduit la charge cognitive, autorise le partage de types entre client et serveur et homogénéise l'outillage. NestJS apporte une structure d'entreprise (modules, providers, guards) qui anticipe la montée en complexité, tandis que React et Vite offrent une expérience de développement rapide côté client.

5. Modèle de données

Le modèle relationnel est géré par TypeORM sur PostgreSQL/PostGIS. Le diagramme entité-relation complet figure en Annexe B. Les entités sont déclarées par décorateurs, ce qui garantit

la cohérence entre le code et le schéma (synchronisation automatique en développement, migrations en production).

5.1 Entités principales

Entité	Rôle
User	Comptes, rôle RBAC, points, secret MFA, compteur d'échecs/verrouillage
Zone	Secteur géographique (12 secteurs)
Container	Conteneur connecté : position (Point 4326), seuils, état courant
Measurement	Mesure capteur (niveau, température, batterie, horodatage)
Alert	Alerte (type, gravité, statut, cycle de vie)
CollectionRoute	Tournée : arrêts ordonnés (JSONB), distance, durée
Signalement	Remontée citoyen/agent géolocalisée
Badge / UserBadge	Catalogue de badges et badges débloqués
Challenge / ChallengeParticipation	Défis collectifs et participations
Settings	Paramètres globaux (singleton)

5.2 Données géospatiales (PostGIS)

La position des conteneurs est stockée comme géométrie PostGIS (type Point, système de référence WGS84 / SRID 4326) et indexée par un index spatial GiST. Cela permet des requêtes de proximité performantes, utilisées par la fonction « conteneurs à proximité » et par l'optimisation des tournées.

container.entity.ts / containers.service.ts — géométrie et requête spatiale

```
@Index({ spatial: true })
@Column({ type: 'geometry', spatialFeatureType: 'Point', srid: 4326 })
location: Point;

// Requête de proximité (rayon en mètres) — PostGIS ST_DWithin
qb.where(
  `ST_DWithin(c.location::geography,
    ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography, :radius)`,
  { lat, lng, radius });
```

5.3 Stratégies d'optimisation

- Index GiST sur la colonne géométrique des conteneurs.

- Table des mesures indexée par (conteneur, date) pour absorber le volume (~500 000 lignes/jour) ; partitionnement temporel envisagé en évolution.
- Arrêts de tournée stockés en JSONB (structure ordonnée, lecture en un accès).
- Cache Redis des lectures fréquentes (objectif de taux de succès > 80 %).

5.4 Dictionnaire de données (entités principales)

User — comptes et sécurité :

Attribut	Type	Contraintes / description
id	uuid	Clé primaire
email	varchar	Unique, identifiant de connexion
passwordHash	varchar	bcrypt (coût 10), non sélectionné par défaut
role	enum	CITOYEN AGENT GESTIONNAIRE ADMIN
points	int	Points de gamification cumulés
mfaSecret	varchar	Secret TOTP, nullable, non sélectionné par défaut
mfaEnabled	boolean	MFA active
failedLoginAttempts	int	Compteur d'échecs consécutifs
lockedUntil	timestampz	Verrouillage temporaire, nullable

Container — conteneur connecté :

Attribut	Type	Contraintes / description
id	uuid	Clé primaire
code	varchar	Unique (identifiant / QR)
location	geometry(Point,4326)	Position WGS84, index spatial GiST
zoneId	uuid	Clé étrangère vers Zone
capacityLiters	int	Capacité du conteneur
currentFillLevel	int	Dernier niveau connu (0-100 %)

thresholdWarn / thresholdCritical	int	Seuils d'alerte (défaut 70 / 90)
status	enum	OK WARNING CRITICAL UNKNOWN
lastMeasurementAt	timestampz	Horodatage de la dernière mesure

Measurement — mesure capteur (fort volume) :

Attribut	Type	Contraintes / description
id	uuid	Clé primaire
containerId	uuid	FK Container ; index composite (containerId, recordedAt)
fillLevel	int	Niveau de remplissage (0-100 %)
temperature	real	Température, nullable
battery	int	Niveau de batterie du capteur, nullable
recordedAt	timestampz	Date de la mesure

CollectionRoute — tournée optimisée :

Attribut	Type	Contraintes / description
id	uuid	Clé primaire
status	enum	PLANNED IN_PROGRESS COMPLETED
agentId / zoneId	uuid	Agent assigné / zone, nullable
stops	jsonb	Arrêts ordonnés (TSP) : code, lat/lng, niveau, collecté
totalDistanceMeters	int	Distance totale calculée
estimatedDurationMin	int	Durée estimée (conduite + service)

Signalement — remontée citoyen/agent :

Attribut	Type	Contraintes / description
id	uuid	Clé primaire
citizenId	uuid	Auteur (FK User)
containerId	uuid	Conteneur concerné, nullable
type	varchar	CONTENEUR_PLEIN DEPOT_SAUVAGE CONTENEUR_ENDOMMAGE
photoUrl	varchar	Photo jointe, nullable
lat / lng	double	Position du signalement
status	enum	NEW IN_REVIEW RESOLVED

6. Conception des API REST

L'API expose 40 opérations documentées automatiquement au format OpenAPI/Swagger (consultables sur `/api/docs` ; export complet en Annexe C).

6.1 Conventions

- Ressources au pluriel, verbes HTTP porteurs de sémantique (GET, POST, PATCH, PUT, DELETE).
- Préfixe global `/api` ; authentification par jeton Bearer (JWT).
- Validation stricte des entrées via class-validator (mode whitelist).
- Codes HTTP normalisés : 200/201, 400/422 (validation), 401 (non authentifié), 403 (interdit/verrouillé), 404, 429 (rate limit).

6.2 Exemple : contrôleur et DTO

Validation déclarative des entrées et restriction par rôle

```
export class CreateContainerDto {
  @IsString() code: string;
  @Type(() => Number) @IsLatitude() lat: number;
  @Type(() => Number) @IsLongitude() lng: number;
  @IsOptional() @IsInt() @Min(1) @Max(100) thresholdWarn?: number;
}

@Post()
@UseGuards(RolesGuard)
@Roles(Role.GESTIONNAIRE, Role.ADMIN)
create(@Body() dto: CreateContainerDto) {
  return this.containers.create(dto);
}
```

6.3 Endpoints représentatifs

Méthode	Endpoint	Description
POST	/api/auth/login	Connexion (tokens ou défi MFA)
GET	/api/containers	Liste des conteneurs (filtres zone/état)
GET	/api/containers/nearby	Conteneurs dans un rayon (PostGIS)
POST	/api/routes/optimize	Optimisation d'une tournée (TSP + 2-opt)
POST	/api/routes/{id}/stops/{cid}/collect	Validation d'une collecte
POST	/api/signalements	Signalement citoyen
GET	/api/reports/monthly	Rapport mensuel

6.4 Exemples de requêtes et de réponses

Connexion : réponse standard ou défi MFA

```
POST /api/auth/login
{ "email": "gestionnaire@ecotrack.fr", "password": "Password123" }

200 OK
{ "accessToken": "eyJhbGciOiJIUzI1NiI... ",
  "refreshToken": "eyJhbGciOiJIUzI1NiI... " }

// Si la MFA est activée :
200 OK
{ "mfaRequired": true, "mfaToken": "eyJJ..." }
```

Optimisation d'une tournée (TSP) : réponse

```
POST /api/routes/optimize
{ "minFillLevel": 70 }

201 Created
{ "id": "...", "status": "PLANNED",
  "stops": [ { "order": 1, "code": "CT-0042", "lat": 45.77, "lng": 4.83,
    "fillLevel": 88 }, ... ],
  "totalDistanceMeters": 43800, "estimatedDurationMin": 272 }
```

6.5 Format normalisé des erreurs

Les erreurs suivent le format standard de NestJS : code HTTP, libellé et message explicite.

Exemples de réponses d'erreur


```

401 Unauthorized
{ "statusCode": 401, "message": "Identifiants invalides.", "error":
  "Unauthorized" }

403 Forbidden (compte verrouillé)
{ "statusCode": 403, "message": "Compte temporairement verrouillé...", "error":
  "Forbidden" }

422 Unprocessable Entity (validation)
{ "statusCode": 422, "message": ["lat must be a latitude string or number"] }

```

7. Sécurité

La sécurité a été traitée dès la conception (security by design), en regard de l'OWASP Top 10 et des exigences du RGPD.

7.1 Authentification, sessions et MFA

- Jetons JWT : access (15 min) et refresh (7 jours) avec rotation ; le hash du refresh token est stocké côté serveur, permettant sa révocation.
- Mots de passe hachés avec bcrypt (coût 10).
- Verrouillage de compte après 5 tentatives échouées (15 minutes).
- Double authentification (MFA) par TOTP (otplib) activable pour gestionnaire et administrateur ; connexion en deux étapes.

auth.service.ts — verrouillage et bifurcation MFA

```

async login(dto: LoginDto): Promise<Tokens | MfaChallenge> {
  const user = await this.users.findByEmailWithSecret(dto.email);
  if (!user) throw new UnauthorizedException('Identifiants invalides.');
```

```

  if (user.lockedUntil && user.lockedUntil > new Date())
    throw new ForbiddenException('Compte temporairement verrouillé.');
```

```

  const valid = await bcrypt.compare(dto.password, user.passwordHash);
  if (!valid) {
    await this.users.registerFailedAttempt(user.id); // verrouille au 5e échec
    throw new UnauthorizedException('Identifiants invalides.');
```

```

  }
  await this.users.resetFailedAttempts(user.id);
  if (user.mfaEnabled) return { mfaRequired: true, mfaToken };
  return this.issueTokens(user);
}

```

7.2 Autorisation (RBAC)

Le contrôle d'accès est appliqué côté serveur par un guard qui compare le rôle de l'utilisateur (extrait du JWT) aux rôles requis déclarés sur la route.

roles.guard.ts — contrôle d'accès basé sur les rôles

```

@Injectable()
export class RolesGuard implements CanActivate {

```

```

    constructor(private reflector: Reflector) {}
    canActivate(ctx: ExecutionContext): boolean {
        const required = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY,
            [ctx.getHandler(), ctx.getClass()]);
        if (!required?.length) return true;
        const { user } = ctx.switchToHttp().getRequest();
        if (!user || !required.includes(user.role))
            throw new ForbiddenException('Droits insuffisants.');
```

```

        return true;
    }
}

```

7.3 OWASP Top 10 — mesures

Risque OWASP	Mesure dans ECOTRACK
A01 Contrôle d'accès défaillant	RBAC côté serveur (guards), vérification systématique
A02 Défaillances cryptographiques	bcrypt (mots de passe), TLS (transport), secrets hors code
A03 Injection	Requêtes paramétrées (ORM), validation stricte des entrées
A05 Mauvaise configuration	CORS contrôlé, en-têtes, variables d'environnement
A07 Authentification défaillante	JWT + refresh + MFA + verrouillage
A09 Journalisation	Logs structurés des accès et erreurs

7.4 Conformité RGPD

- Minimisation : seules les données nécessaires aux signalements sont collectées.
- Finalité limitée à la gestion du service ; pas de revente.
- Sécurité des données personnelles (hachage, chiffrement en transit).

8. Optimisation des tournées (TSP)

Le service d'optimisation résout une instance du problème du voyageur de commerce (TSP) afin de minimiser la distance d'une tournée. La solution combine une heuristique constructive et une amélioration locale.

- Plus proche voisin : construction d'un itinéraire initial depuis le dépôt (centre de la métropole).
- 2-opt : amélioration locale supprimant les croisements (inversion de segments tant que la distance diminue).

- Distances géodésiques calculées par la formule de Haversine.
- Complexité : plus proche voisin en $O(n^2)$, 2-opt en $O(n^2)$ par passe — adapté à des tournées de quelques dizaines de conteneurs (calcul en quelques secondes).

tsp.util.ts — combinaison plus proche voisin + 2-opt

```
export function optimizeRoute(depot: GeoPoint, points: GeoPoint[]) {
  if (points.length === 0) return { order: [], distanceMeters: 0 };
  const nn = nearestNeighbour(depot, points); // solution initiale
  const order = points.length > 2 ? twoOpt(depot, points, nn) : nn; //
  raffinement
  const path = [depot, ...order.map(i => points[i])];
  return { order, distanceMeters: Math.round(totalDistance(path)) };
}
```

Le seuil de remplissage à partir duquel un conteneur entre dans une tournée est paramétrable par le gestionnaire (UC-G01). Le module d'optimisation est couvert à 100 % par les tests unitaires.

9. Temps réel et chaîne IoT

La chaîne IoT reproduit fidèlement un déploiement réel sans matériel physique, autour du broker MQTT.

9.1 Flux de données

Capteur (simulé) → broker MQTT → service d'ingestion → enregistrement de la mesure → calcul de l'état du conteneur → déclenchement d'alerte si seuil franchi → diffusion temps réel (WebSocket) vers le tableau de bord. Chaque message respecte un schéma JSON simple.

Schéma d'un message de mesure IoT (topic ecotrack/measurements)

```
{ "containerId": "...", "fillLevel": 82,
  "temperature": 18.4, "battery": 76,
  "recordedAt": "2026-05-26T09:38:57.000Z" }
```

9.2 Service d'ingestion

ingestion.service.ts — consommation du flux MQTT

```
this.client = mqtt.connect(url, { reconnectPeriod: 5000 });
this.client.on('connect', () => this.client.subscribe(topic));
this.client.on('message', async (_t, payload) => {
  const data = JSON.parse(payload.toString());
  await this.measurements.record(data); // état + alertes + temps réel
});
```

9.3 Traitement et temps réel

À la réception d'une mesure, le service met à jour le niveau et l'état du conteneur (OK / À collecter / Critique), lève une alerte en cas de franchissement de seuil (avec déduplication des alertes ouvertes), puis publie une mise à jour que le tableau de bord reçoit par WebSocket. La carte se met ainsi à jour sans rechargement (pattern Observer).

10. Gamification et engagement citoyen

Le volet participatif vise l'engagement des 15 000 citoyens autour d'un mécanisme de points, badges et défis collectifs.

- Chaque signalement validé crédite 10 points et peut débloquer des badges (paliers).
- Un classement (leaderboard) valorise les citoyens les plus actifs.
- Les défis collectifs fixent un objectif commun ; à son atteinte, l'ensemble des participants est récompensé.
- L'impact environnemental individuel (CO₂ évité) est estimé et restitué au citoyen.

Badge	Palier
Premier geste 🌱	10 points
Éco-citoyen ♻️	50 points
Gardien du quartier 🛡️	150 points
Champion du tri 🏆	300 points

gamification.service.ts — attribution de points et badges

```
async awardPoints(userId: string, delta: number) {
  await this.users.addPoints(userId, delta);
  const user = await this.users.findById(userId);
  const eligible = await this.badges.find({
    where: { threshold: LessThanOrEqual(user.points) } });
  for (const badge of eligible) // débloquent les badges atteints
    if (!await this.userBadges.findOne({ where: { userId, badgeId: badge.id } }))
      await this.userBadges.save({ userId, badgeId: badge.id });
}
```

11. Stratégie de tests et qualité

La qualité repose sur une suite de tests automatisés (Jest) et des outils d'analyse statique (ESLint, Prettier). La suite compte 50 tests unitaires répartis en 13 suites.

11.1 Couverture mesurée

La couverture est mesurée sur les services métier et les utilitaires (logique applicative). Les adaptateurs d'entrée/sortie (simulateur et ingestion MQTT, script de seed) sont exclus du périmètre de couverture unitaire ; ils relèvent de tests d'intégration.

Périmètre	Couverture lignes
-----------	-------------------

Global (services + utilitaires)	≈ 82 %
Optimisation TSP	100 %
Mesures (pipeline IoT)	≈ 91 %
Tournées	≈ 86 %
Signalements	≈ 86 %
Gamification	≈ 82 %
Authentification (lockout/MFA)	≈ 64 %
Cible CDC	> 60 %

11.2 Approche pyramidale

- Base : tests unitaires des services avec dépôts et dépendances simulés (mocks).
- Cas de sécurité couverts : verrouillage après 5 échecs, défi MFA, vérification TOTP invalide/ valide.
- Sommet (perspective) : tests end-to-end (Playwright) et intégration au pipeline CI.

auth.service.spec.ts — extrait de test de sécurité

```
it('exige la MFA si activée', async () => {
  const { service } = build({ mfaEnabled: true });
  mockedBcrypt.compare.mockResolvedValue(true);
  const res = await service.login({ email, password });
  expect(res.mfaRequired).toBe(true);
});
```

12. Infrastructure, conteneurisation et déploiement

L'ensemble de la plateforme est conteneurisé et démarrable d'une seule commande (docker compose up).

12.1 Composition Docker

Service	Image / Rôle
postgres	PostgreSQL 15 + PostGIS (persistance)
redis	Cache et rate limiting
mosquitto	Broker MQTT (ingestion IoT)
backend	API NestJS
frontend	SPA React (Vite en dev, Nginx en production)

12.2 Image de production multi-étapes

backend/Dockerfile — build multi-étapes (image finale légère)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build && npm prune --production

FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
CMD ["node", "dist/main.js"]
```

12.3 Environnements et déploiement

- Développement : hot-reload via volumes montés ; variables dans un fichier .env.
- Production : images multi-étapes ; frontend servi par Nginx.
- Cible de déploiement : frontend sur Vercel, backend et base managée sur Railway (URL publique).
- Données de démonstration injectées automatiquement au premier démarrage (12 secteurs, 144 conteneurs, comptes de démonstration).

13. Exigences non fonctionnelles (NFR)

Exigence	Cible	Mise en œuvre
Temps de réponse API	< 500 ms (p95)	Cache Redis, requêtes indexées
Requêtes SQL	< 200 ms (90 %)	Index GiST, index composites
Chargement page	< 3 s	Vite, code splitting
Utilisateurs simultanés	500	Architecture sans état, scalable
Taux de succès cache	> 80 %	Redis
Disponibilité	99,5 %	Conteneurs supervisés, redémarrage auto
Accessibilité	WCAG 2.1 AA (amorce)	Statut par icône + texte + couleur, focus, contrastes
Conformité	RGPD	Minimisation des données personnelles

13.1 Éco-conception (numérique responsable)

- Réduction des tournées (–20 %) = réduction directe des émissions de CO₂.
- Optimisation des requêtes et mise en cache : moins de calcul et de transfert réseau.
- Mutualisation des conteneurs Docker et images légères : empreinte d’infrastructure réduite.
- Rafraîchissement temps réel par événements (WebSocket) plutôt que sondage périodique.

14. Gestion de projet

Le projet a été conduit en méthodologie Scrum, sur un horizon de quatre mois.

Jalon	Période	Livrables
Cadrage (Sprint 0)	S1-3	Architecture, environnement, backlog
Socle et modules cœur	S4-7	Auth, conteneurs, mesures, carte
Fonctionnalités avancées	S8-11	TSP, gamification, défis, mobile/agent
Intégration et tests	S12-14	Tests, sécurité (MFA), polish

Soutenance	S15-16	Présentation, démonstration, DCT
-------------------	--------	----------------------------------

- Outils : Git/GitHub (versionnage), tableau de backlog, documentation technique.
- Compétences transversales : veille et documentation en anglais technique ; éco-conception intégrée.

15. Conclusion et perspectives

ECOTRACK constitue une implémentation complète et fonctionnelle du cahier des charges : les quatorze cas d'utilisation sont couverts et validés, l'application est conteneurisée, testée ($\approx 80\%$ de couverture des services) et documentée. Elle démontre la maîtrise d'une chaîne full-stack moderne intégrant géospatial, temps réel, IoT, sécurité (dont MFA) et engagement citoyen.

Les perspectives d'évolution prolongent l'architecture actuelle :

- Bus d'événements distribué (Apache Kafka) pour l'ingestion à très grande échelle (exactly-once, partitions).
- Extraction des modules en microservices physiques orchestrés par Kubernetes (Helm, autoscaling).
- Observabilité complète (Prometheus, Grafana, traçabilité distribuée OpenTelemetry).
- Modèle prédictif de remplissage (MLOps) pour anticiper les collectes.

Annexes

Annexe A — Schéma d'architecture

Les diagrammes d'architecture sont fournis au format Mermaid dans le fichier annexes/architecture.md (rendu sur mermaid.live, sur GitHub ou via VS Code, puis export PNG/SVG).

Trois vues sont fournies :

- A.1 — Vue d'ensemble en conteneurs (type C4 niveau 2) : clients React, API NestJS et ses modules, PostgreSQL/PostGIS, Redis, broker MQTT, passerelle WebSocket, périmètre Docker Compose.
- A.2 — Diagramme de séquence du flux temps réel : du capteur (MQTT) jusqu'à la mise à jour du tableau de bord (WebSocket), avec déclenchement d'alerte.
- A.3 — Diagramme de séquence de l'optimisation de tournée (TSP) et de la validation de collecte par l'agent (scan QR).

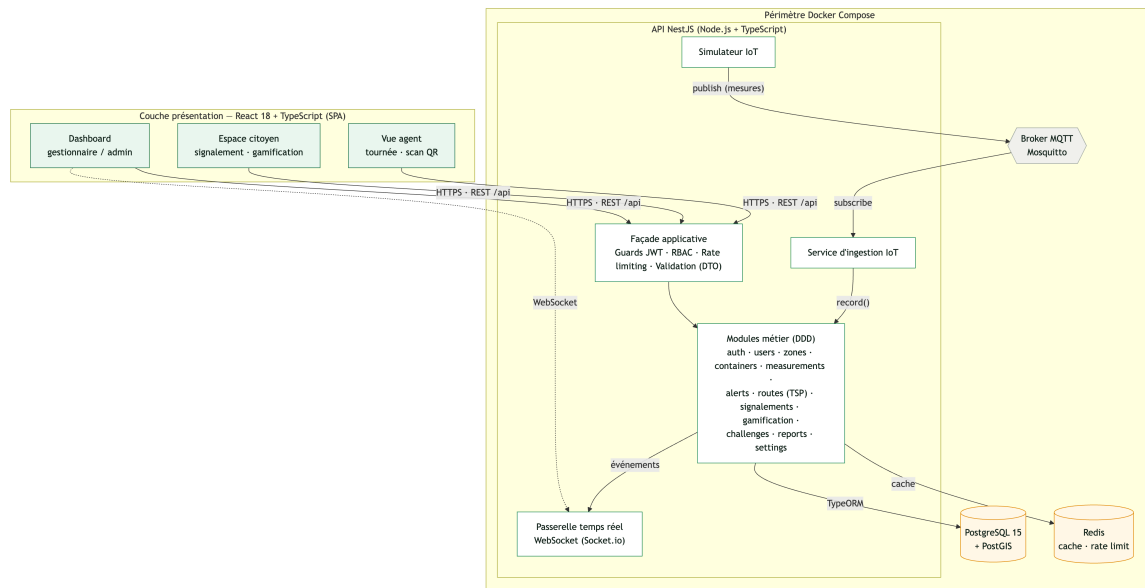


Figure A.1 — Vue d'ensemble de l'architecture (C4 niveau 2)

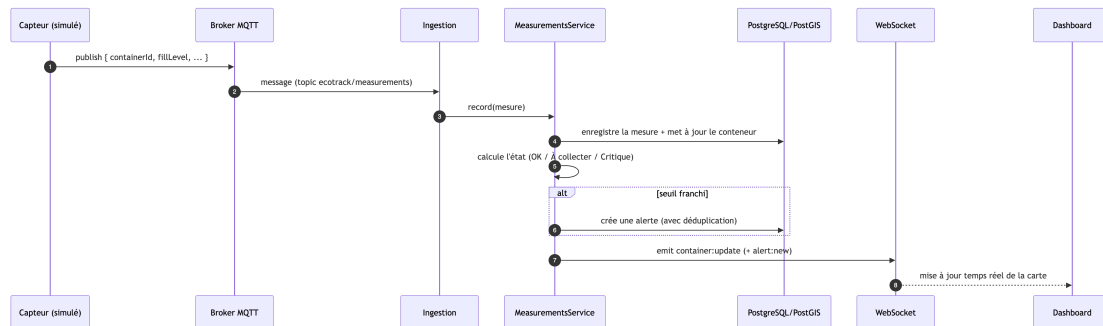


Figure A.2 — Flux temps réel : du capteur IoT au tableau de bord

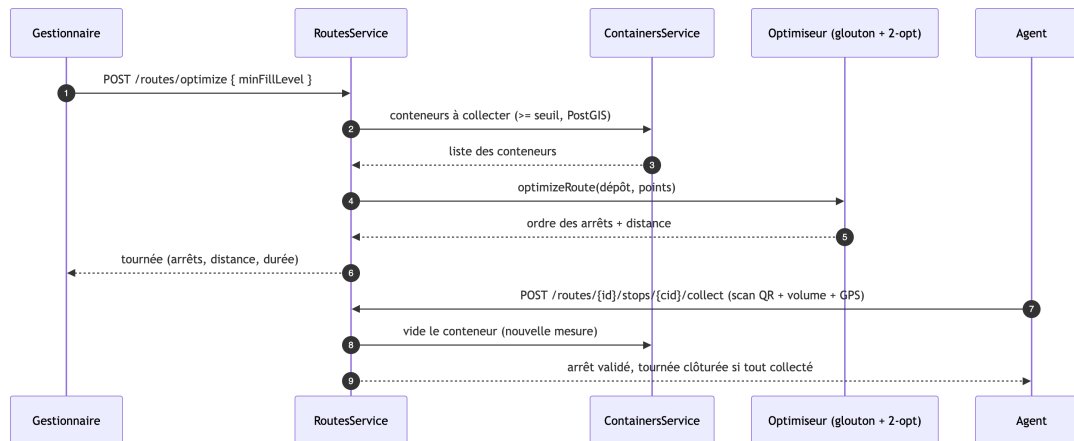


Figure A.3 — Optimisation de tournée et validation de collecte

Annexe B — Modèle de données (ERD)

Diagramme entité-relation complet dans le fichier annexes/erd.md (format Mermaid, rendu sur mermaid.live ou GitHub).

Annexe C — Documentation API (OpenAPI)

Spécification OpenAPI complète exportée dans annexes/openapi.json (40 opérations), consultable sur /api/docs (Swagger UI).

Annexe D — Captures d'écran

[Insérer les captures : tableau de bord et carte temps réel, espace citoyen et gamification, vue agent avec scan QR, écran d'administration, rapport mensuel, double authentification.]

ECOTRACK
LYON MÉTROPOLE

Pilotage | Tournées | Citoyen

Gilles Gestion
GESTIONNAIRE | Quitter

Pilotage de la collecte

Vue temps réel des conteneurs de la métropole.

CONTENEURS	À COLLECTER	CRITIQUES	REMPLISSAGE MOY.	ALERTES	SIGNALEMENTS
144	55	23	53%	78	4

Tournée optimisée
Algorithme TSP + 2-opt sur les conteneurs à collecter.
Seuil de collecte: 70%
Générer une tournée

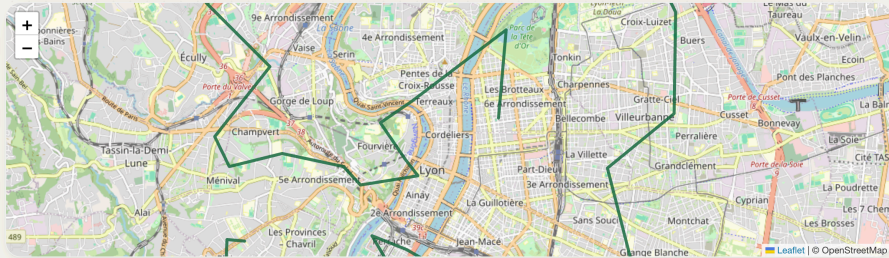
Rapport mensuel
Synthèse exportable en PDF (UC-G03).
May 2026 | PDF

Sécurité — MFA
Recommandée pour les comptes gestionnaire et agent.

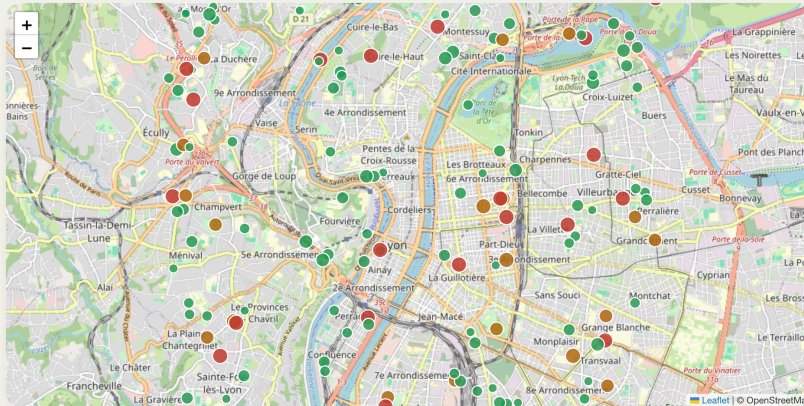
Itinéraires de collecte optimisés.

Scanner un QR ✓ Clôturer

29 mai, 12:22 Planifiée

**Valider**

Signalez un problème, gagnez des points, grimpez au classement.



Touchez la carte pour situer le signalement.



Détails (optionnel)

Choose File No file chosen

[Envoyer le signalement](#)

MES POINTS

CO₂ ÉVITÉ

Prochain badge 🌱 Premier geste à 10 pts



Annexe E — Glossaire

Terme	Définition
MQTT	Protocole de messagerie léger pour l'IoT (publication/abonnement)
PostGIS	Extension géospatiale de PostgreSQL
TSP	Problème du voyageur de commerce (optimisation d'itinéraire)
2-opt	Heuristique d'amélioration locale supprimant les croisements
JWT	Jeton d'authentification signé (JSON Web Token)
RBAC	Contrôle d'accès basé sur les rôles
MFA / TOTP	Double authentification / code à usage unique basé sur le temps
WebSocket	Canal bidirectionnel temps réel client-serveur
DTO	Objet de transfert de données (validation des entrées)
ORM	Mapping objet-relationnel (TypeORM)
SPA	Application web monopage
CI/CD	Intégration et livraison continues